

Constimator — Client Requirements

Implementation Playbook

Audience: Engineering (implement in Cursor / Claude)

Date: 5 September 2026

Source of requirements: Prime contractor conversation (Mark, Raposo Engineering) — pain points that map to the product roadmap

Product: Constimator — bid estimating and bid-form reconciliation for civil / roadway contractors

This document is the detailed, ordered to-do list. Do **List 1 first**. Each numbered item is one Cursor task: paste the heading plus Current / Change / Files / Done when into a new chat.

How to use this document

Work **in order**. Do not start List 3 or List 4 until List 1 is demoable on a real bid-form PDF.

Each checkbox is a single implementation unit. House rules for every list:

- Read `node_modules/next/dist/docs/` before changing App Router or Server Actions. This Next.js version has breaking changes versus older training data.
- Org-scoped data only through `getScopedDb()` in `lib/db/scoped.ts`. Never import the raw Drizzle client from app code.
- New tables need `orgId`, indexes, `orgIsolationPolicy(...)`, and `.enableRLS()`. Then `pnpm db:generate` and `pnpm db:migrate`.
- Validate Server Action input with zod and `parseInput` from `lib/validation.ts`.
- Worker types in `worker/src/types.ts` must be hand-copied to `lib/cost-engine/types.ts`. There is no shared package.
- Do not skip the human confirm click on anything the AI extracted.

Recommended calendar (AI-assisted, ~3 weeks)

Week	Work	What you can show a prime
1	List 1.1–1.7	Upload bid form → reconciliation is populated
1–2	List 1.8	Same flow proven on 3–5 real agency PDFs

Week	Work	What you can show a prime
2	List 2	Plan holders + leveled quotes (not “who to list” legally)
3	List 4 or List 3	Spec checklist or email invites — whichever the next contractors repeat

What we are *not* building in this pass

Client ask	Hard-AI / later version — do not start
Bid-form auto-fill	Silent auto-import with no review; Excel/Word forms; measuring quantities off drawings
Who to list	CSLB/DIR/DBE registry, “list these three firms,” listing-threshold as a legal answer
Bid invites	Sub portal, magic links, SMS, CSV/XLSX plan-room ingest
Spec bid/no-bid	An AI that says “bid this / pass this”; live RFI generation from plans

LIST 1 — Bid-form auto-fill into reconciliation

Goal

Upload Official Bid Form → AI transcribes the schedule → import card is the main action → `bid` rows exist → reconciliation diffs against the estimate. The contractor does **not** retype 80 line items.

Already built — do not recreate

Piece	Path
Extractor	<code>worker/src/extract-bid-form.ts</code> → <code>extractBidForm()</code>
Job routing	<code>worker/src/process-job.ts</code> when <code>document.type === "bid_form"</code>
Pending detection	<code>lib/bid-form-import.ts</code> → <code>pendingBidFormExtractions()</code>
Import UI	<code>components/reconciliation/bid-form-import-card.tsx</code>
Import write	<code>app/reconciliation/actions.ts</code> → <code>importExtractedBidFormAction()</code>
Estimate seed	<code>app/estimate/actions.ts</code> → <code>importFromBidScheduleAction()</code>
Unit tests	<code>lib/bid-form-import.test.ts</code>
Analytics	<code>"bid_form_imported"</code> already in <code>lib/analytics.ts</code>
Real-PDF harness	<code>scripts/real-job/</code>

Critical routing fact: If the uploaded file is *not* typed Official Bid Form (`bid_form`), the worker runs plan-sheet takeoff instead. `pendingBidFormExtractions` then ignores it because `kind !== "bid_form"`.

1.1 — Empty state must push import, not typing

Current

`components/reconciliation/reconciliation-shell.tsx` (approximately lines 151–171). If `pendingExtractions.length > 0`, the import card shows, **then** the empty state still says “Enter the official bid form’s line items below.” That tells people to type even when extraction is ready.

Change

- If `pendingExtractions.length > 0` and `!hasBidForm`:
 - Title: **Extracted bid form ready to import**
 - Description: Review the table, then import. You can edit any row after.
 - Do **not** show “Enter line items” as the primary message.
- If no pending extraction and no bid form:
 - Title: **No official bid form yet**
 - Description: Upload the official bid form under Upload Documents (type Official Bid Form). When extraction finishes, import it here. Typing rows is only a fallback.
 - Button: `router.push(\ /upload?project=${projectId})`` labeled **Upload bid form**.
- Keep `BidLineTable` below as fallback in both cases.

Files

- `components/reconciliation/reconciliation-shell.tsx`

Done when

With a completed bid-form job and zero `bid` rows, the first thing you see is the import card, not “type it in.”

1.2 — Import preview: confidence, notes, page, skip rows

Current

`bid-form-import-card.tsx` shows `#`, Description, Unit, Qty only. It always imports `extraction.items` in full. `extractedBidItemSchema` already has `confidence`, `sourcePage`, `notes`. `bid.extraction_confidence` is stored on import but never shown.

Change in `components/reconciliation/bid-form-import-card.tsx`

1. Local state: `skippedItemKeys: Set<string>`. Key = ``${itemNumber}::${description}`` (same as today’s table key — if two rows collide, use index).
2. Table columns: **# | Description | Unit | Qty | Conf | Notes**. Optional page as `p.{sourcePage}` under the description.
3. Confidence cell:
 - `null` → “—”

- `< 90` → warning color (`text-warning`), same cutoff as `LOW_CONFIDENCE = 90` in `components/plan-holders/holder-row.tsx`
 - `>= 90` → muted tabular number
4. Checkbox (or Skip) per row. Skipped rows are excluded from the action payload.
 5. Import button label: `Import N items` where `N` = not skipped. Disable if `N === 0`.
 6. Pass only selected items into `importExtractedBidFormAction`.

Change in `app/reconciliation/actions.ts`

`importExtractedBidFormAction` already takes `items`. No schema change required if the client sends the filtered list. Optionally add `skippedCount` to the `bid_form_imported` analytics properties.

Tests in `lib/bid-form-import.test.ts`

Add `filterSkipped(items, skippedKeys)` in `lib/bid-form-import.ts` so filtering does not live only in React.

Cases:

- All selected
- One skipped
- All skipped returns `[]`

Done when

An 88-confidence row is visually flagged; unchecking it means it is not in `bid` after import.

1.3 — Show extraction confidence on the official schedule after import

Current

`components/reconciliation/bid-line-table.tsx` `openEdit` does not pass `extractionConfidence`. The table has no confidence column. Schema column already exists: `bids.extractionConfidence` in `db/schema.ts` (approximately line 735).

Change

- Add a narrow **AI conf** column after Qty.

- Same `< 90` warning styling as the import card.
- Manual rows (`documentId` null, confidence null) show “—”.
- Do **not** add a notes column to `bid` in this task. Confidence-only is enough. Notes are not stored on `bid` today.

Files

- `components/reconciliation/bid-line-table.tsx`

Done when

Imported rows show 94 / 88 / etc.; typed rows show —.

1.4 — Optional “also seed the estimate” after import

Current

Estimate Workspace has **Import from Bid Schedule** (`components/estimate/estimate-shell.tsx` approximately 269–277) calling `importFromBidScheduleAction` . It creates estimate lines with `source: "official"` , `quantity = official quantity` , `unitPrice: "0"` . It skips bids already linked by `bidId` or a unique description match.

Change

- After successful import in `BidFormImportCard` , show a secondary button: **Create estimate lines from this schedule.**
- Call `importFromBidScheduleAction(projectId)` .
- Toast: Added N estimate lines at \$0 – price them in Estimate Workspace.
- If `added === 0` , toast: Estimate already has those items.
- Do **not** auto-seed without a click. Zero-price lines look like a real estimate.

Files

- `components/reconciliation/bid-form-import-card.tsx` (import the estimate action)

Done when

Import → click seed → `/estimate` has one line per bid item at \$0, linked by `bidId` .

1.5 — Processing / upload copy so people type the form as Official Bid Form

Current

`components/upload/upload-shell.tsx` checklist includes Official Bid Form. E2E (`e2e/full-flow.spec.ts`) uploads the sample PDF as Supporting Document. If type is not `bid_form` , `process-job.ts` runs **plan takeoff**, not bid-form transcription.

Change

- On Upload, if the user has not marked any file Official Bid Form and they click continue, keep the existing warning.
- After processing completes, if a `bid_form` job is complete, add a one-line on `/processing` : **Bid form extracted — open Bid Reconciliation to import**. Find the complete-state UI in `components/processing/processing-shell.tsx` .
- Do not change the E2E fixture type in this task unless you add a separate bid-form e2e (see 1.8).

Files

- `components/processing/processing-shell.tsx`
- `app/processing/actions.ts` if you need `kind` on the job result (it likely already has it)

Done when

A correctly typed bid-form PDF ends with a clear next step to `/reconciliation` .

1.6 — Marketing / pilot copy (so demos match the product)

Current

`components/how-it-works.tsx` comment says there is no upload-and-auto-fill; step 01 is “Enter the official bid form” item by item. `docs/PILOT_CHECKLIST.md` step 5 still leads with typing.

Change

- How it works 01: **Upload the official bid form**. Constimator reads the schedule; you confirm and import. Typing is only if you have no PDF.

- FAQ `components/faq.tsx` “What documents does it need?”: official bid form is uploaded and imported, not only typed.
- Pilot step 5: import after review is primary; typing is fallback.

Files

- `components/how-it-works.tsx`
- `components/faq.tsx`
- `docs/PILOT_CHECKLIST.md`

Done when

A contractor reading the site is not told to retype 80 lines.

1.7 — Unit tests for pending / import mapping

Current tests

Pending job returned; skip if `documentId` already on a bid; ignore `plan_takeoff` ; map fields including confidence.

Add

- Latest job wins if two jobs exist for one document (function already sorts by `createdAt` desc — assert it).
- Failed / queued jobs ignored.
- Empty `bidItems` → not pending.
- Invalid item (missing description) → whole parse fails, document not pending (current `safeParse` on the array). Do **not** silently drop rows without a warning.
- `bidRowsFromExtractedItems` : missing confidence → `extractionConfidence: null` .

File

- `lib/bid-form-import.test.ts`

Run

```
pnpm test lib/bid-form-import.test.ts
```

1.8 — Real-PDF validation (required before you tell a client it is reliable)

Harness: `scripts/real-job/README.md`

Steps

1. Worker running: `cd worker && npm run dev`
2. Upload a real bid-form PDF:

```
node --env-file=.env.local scripts/real-job/upload.ts ./scripts/real-job/pdfs/<file>.pdf --type bid_form
```

3. Key the answer CSV under `scripts/real-job/expected/` (gitignored).
4. Compare:

```
node --env-file=.env.local scripts/real-job/compare.ts --document <id> --expected ./scripts/real-job/expected/<name>.csv
```

5. Repeat for **at least 3** real civil / DOT / county forms (clean digital PDFs).
6. Record: row count, MISSING, EXTRA, QTY, UNIT, mean confidence on ok vs wrong.

If things go wrong

- **Long schedules truncate mid-list:** raise `max_tokens` in `worker/src/extract-bid-form.ts` (currently 16000) and re-run. Do not switch to silent auto-import.
- **File uploaded as Plans / Other:** re-upload with type Official Bid Form. Do not feed takeoff items into `bid`.

Done when

You have a short note of 3 packages: importable, and wrong rows are low-confidence or noted.

List 1 — out of scope

- Auto-import with no confirm
- Excel / Word bid forms
- Plan-sheet quantity takeoff as the official schedule
- Per-cell edit in the import preview (edit after import is enough)

LIST 2 — “Who to list” demo (plan holders + quote leveling)

Goal

A prime can (a) see who pulled the plans and (b) see who quoted a trade, leveled, cheapest after exclusions. Copy must say this is **not** a listing-form recommendation.

Already built

- Upload / review plan holders: `app/plan-holders/` , `components/plan-holders/`
- Extractor: `worker/src/extract-plan-holders.ts`
- Quote upload / review / compare: `app/sub-quotes/` , `lib/quote-comparison.ts` , `components/sub-quotes/comparison-grid.tsx`
- Registry is a **seam only**: `planHolderMatchStatusEnum` , `contractorId` always unmatched. See `docs/REGISTRY-SOURCES.md` . Do not build the registry here.

There is **no** Confirm-all action today (`review-shell.tsx` only has Open source PDF). Add it in 2.2.

2.1 — Plan holders empty / pending copy

Current

`app/plan-holders/page.tsx` uses `NoProjectState` / “No plan holders list yet.” `review-shell.tsx` `PENDING_COPY` covers extracting / failed / none_found.

Change

- No list: button **Upload a plan holders list** → `/plan-holders/upload` .
- Failed: **Retry from AI Processing** link to `/processing` plus **Upload again**.
- `none_found` : keep extractor notes (`review.list.documentNotes`).

Files

- `app/plan-holders/page.tsx`

- `components/plan-holders/review-shell.tsx`
-

2.2 — Confirm remaining (not confirm-all blindly)

Current

Only per-row Confirm in `holder-row.tsx` → `confirmPlanHolderAction`. List status flips to `confirmed` only when every row is confirmed (`refreshListStatus` in `app/plan-holders/actions.ts`).

Change

1. New action `confirmRemainingPlanHoldersAction(listId)` :
 - Load contacts for that list via scoped db.
 - Set `isConfirmed: true`, `confirmedBy`, `confirmedAt` on rows where `isConfirmed === false`.
 - Call existing `refreshListStatus`.
 - Confirm remaining, but toast: **N rows were under 90% confidence — spot-check those.**
2. Button on `review-shell.tsx` next to Open PDF: **Confirm remaining (N)**.
3. Disable when `progress.confirmed === progress.total` or `pendingReason` is set.

Files

- `app/plan-holders/actions.ts`
- `components/plan-holders/review-shell.tsx`

Security

`listId` must belong to the current org (scoped `findFirst` first, throw if missing) — same pattern as other actions.

Done when

A 40-row list can be confirmed in one click; low-confidence rows still visually flagged on `HolderRow`.

2.3 — Export confirmed holders as CSV

Why

The prime will paste this into Excel / a listing worksheet. No registry needed.

Change

- Client-side CSV from already-loaded `contacts` (no new API if data is on the page).
- Columns: `companyName`, `contactName`, `email`, `phone`, `licenseNumber`, `city`, `state`, `isConfirmed`.
- Filename: `plan_holders_{projectNumber}_{sourceLabel}.csv` — sanitize the label.
- Export **confirmed only**, or a toggle confirmed / all. Default confirmed.
- If zero confirmed, disable + tooltip **Confirm rows first**.

Files

- New `lib/plan-holder-export.ts` (pure functions)
- Button in `components/plan-holders/review-shell.tsx`
- Pattern to copy: `lib/comparison-export.ts`

Tests

One vitest: given two contacts, only confirmed appears; commas in company names are quoted.

Done when

Download opens in Excel with emails intact.

2.4 — Sidebar: one “Subs” group so the four links are a flow

Current

`components/app-sidebar.tsx` `mainNav` is a flat list: Upload Sub Quotes, Review Sub Quotes, Compare Quotes, Upload Plan Holders, Review Plan Holders — easy to look unfinished.

Change

Split `mainNav` into groups (`SidebarGroupLabel` already exists):

1. **Documents:** Dashboard, Projects, Upload, Intelligence, Schedules, Cost Setup
2. **Subcontractors:** the five sub / plan-holder links
3. **Bid:** Estimate, Reconciliation, Review, Reports

Do not add new routes.

Done when

A demo can say “this block is subs.”

2.5 — Compare quotes: one-quote and “this is not who to list”

Current

`app/sub-quotes/compare/page.tsx` — if `trades.length === 0`, empty. If one quote exists, the grid still renders one column. Ranking copy lives in `comparison-grid.tsx`.

Change

- If the selected trade has `columns.length < 2`, banner: **Need at least two quotes on this trade to level. Upload another under Upload Sub Quotes.**
- Still show the single quote (conditions are useful).
- Near adjusted-rank #1: **Cheapest after exclusions you costed — not a recommendation of who to list on the bid.**
- Keep the unverified banner (`countUnverified`, `unverifiedNotice`).

Files

- `components/sub-quotes/comparison-grid.tsx`
- Possibly `app/sub-quotes/compare/page.tsx`

Done when

You can demo two quotes and the disclaimer is on screen without you saying it.

2.6 — “Subs on this job” summary

Place: Intelligence Overview (keep the dashboard light).

Data (server, current project)

- Plan holder lists: count, confirmed contacts / total (`app/plan-holders/actions.ts` already has list summaries with `confirmedCount` / `holderCount`).
- Sub quotes: count by trade from `listTradesForComparison` .

UI

Card **Subcontractors** with two lines + links to `/plan-holders` and `/sub-quotes/compare` .

Files

- `app/intelligence/actions.ts` (extend return type)
- `components/intelligence/overview-tab.tsx`

Do **not** invent a recommended-sub list.

List 2 — out of scope

- CSLB / DIR ingest, `matchStatus` other than `unmatched`
 - DBE firm search
 - Listing-threshold % as a legal answer
 - Any “List these companies” ranking
-

LIST 3 — Bid invites to subcontractors

Goal

Select confirmed plan holders who have email → send one Resend email each → persist sent / failed.

Do not reuse `invites` / `inviteTeammateAction` (`db/schema.ts` around line 356). That is org seat invites via Supabase Auth.

Reuse: `getResendClient()` , `EMAIL_FROM` from `lib/email/client.ts` . Pattern: `lib/email/review-request.ts` .

3.1 — Schema

New enum `bid_invite_status` : `sent` , `failed` . (No `draft` in v1.)

New table `bid_invite` :

Column	Type	Notes
<code>id</code>	<code>uuid pk</code>	<code>defaultRandom()</code>
<code>orgId</code>	<code>uuid not null</code>	FK org cascade
<code>projectId</code>	<code>uuid not null</code>	FK project cascade
<code>planHolderContactId</code>	<code>uuid null</code>	FK <code>plan_holder_contact</code> <code>onDelete: set null</code>
<code>email</code>	<code>text not null</code>	snapshot at send time
<code>companyName</code>	<code>text not null</code>	snapshot
<code>trade</code>	<code>text null</code>	optional
<code>message</code>	<code>text null</code>	optional custom note, max ~2000
<code>status</code>	<code>enum</code>	<code>sent</code> / <code>failed</code>
<code>error</code>	<code>text null</code>	Resend / API message
<code>sentAt</code>	<code>timestampz null</code>	set on sent
<code>sentBy</code>	<code>uuid</code>	FK user set null

Column	Type	Notes
createdAt / updatedAt	timestampz	default now

Indexes: `org_id`, `project_id`, `(project_id, email)`.

RLS: `orgIsolationPolicy("bid_invite", table.orgId)`.

Then

1. Edit `db/schema.ts` (follow `plan_holder_contact` comment style).
2. `pnpm db:generate`
3. Read the generated SQL — confirm `ENABLE ROW LEVEL SECURITY` + policy.
4. `pnpm db:migrate`

Wire scoped db

`lib/db/scoped.ts` — import `bidInvites`, add `bidInvites: orgScoped(...)`.

Analytics

Add `"bid_invites_sent"` to `AnalyticsEvent` in `lib/analytics.ts` with properties `{ projectId, sent, failed, skippedNoEmail }`.

3.2 — Email helper

New file `lib/email/bid-invite.ts` (`server-only`):

- Input: `{ to, replyTo, projectName, projectNumber, owner, bidDate, trade, message, primeCompany }`
- Plain `text` body only (HTML later).
- Subject: `Bid invitation — {projectName} ({projectNumber})`
- Body: who is inviting, project, bid date, trade if any, custom message, **Reply to this email with your quote. Do not click unknown links.**
- If `!getResendClient()` throw a user-facing Error (unlike `spend-cap` which no-ops — here the user clicked Send).
- Return Resend id or throw.

Env: `RESEND_API_KEY`, `EMAIL_FROM` must be a verified domain. Document in `.env.example` one line: also used for bid invites.

3.3 — Server action

New `app/plan-holders/invite-actions.ts` (keep `actions.ts` from growing), or add to `actions.ts` if you prefer one file.

```
sendBidInvitesAction({ listId, contactIds: string[], trade?: string, message?: string
}):
```

1. Zod: `contactIds` min 1 max 50; `trade` max 80; `message` max 2000.
2. `getScopedDb()` , load list, assert `list.projectId` is the current project.
3. Load contacts `inArray(id, contactIds)` AND `planHolderListId = listId` . Drop ids that are not in org.
4. Split: no email / has email. No-email → skip, return count.
5. **Require** `isConfirmed` so you do not mail a bad parse.
6. For each with email: try send; insert `bid_invite` `sent` or `failed` . Do not abort the whole batch on one failure.
7. `captureEvent("bid_invites_sent", ...)` .
8. Return `{ sent, failed, skippedNoEmail, skippedUnconfirmed }` .

Rate: 50 sequential Resend calls is OK for v1. Do not send from the browser.

3.4 — UI on Review Plan Holders

`holder-row.tsx`

Optional checkbox. Only enable if `email` is present and `isConfirmed` . If email missing, checkbox disabled + title **No email on the roster**.

`review-shell.tsx`

- Lift selected ids (checkbox state in parent, or a small client wrapper).
- Sticky bar: **N selected + Send bid invite...**
- Dialog: Trade (optional input), Message (textarea), list of recipient emails, warning if some selected rows skipped.
- Submit → action → toast `Sent 12, 1 failed, 3 had no email` → `router.refresh()` .

Sent log

Below the list, table of this project's `bid_invite` rows (newest first): company, email, trade, status, time, error. Load in `app/plan-holders/page.tsx` via new

```
listBidInvitesForProject(projectId) .
```

Done when

Confirm a holder with email → send → they get mail from `EMAIL_FROM` → row status `sent` . Failures show `error` in the log.

3.5 — Tests

- Pure: skip unconfirmed / no email. Extract `partitionInvitees(contacts, selectedIds)` in `lib/bid-invite.ts` .
 - Do not mock Resend in unit tests; keep send in the helper.
 - E2E optional; manual with your own inbox is enough for v1.
-

List 3 — out of scope

- Sub portal / magic links
 - SMS
 - Open / click tracking
 - CSV / XLSX plan-holder upload
 - Reusing teammate `invite` table
-

LIST 4 — Spec checklist (not bid/no-bid AI)

Goal

Specs upload → extract printed go/no-go **facts** → Bid Requirements tab shows them with verbatim quote + page. The product never says “bid this.”

Already built

Participation goals + links only.

- `worker/src/extract-participation-goals.ts`
- `worker/src/select-spec-pages.ts`
- `getIntelligenceData()` maps goals
- Bid Requirements tab (`components/intelligence/bid-requirements-tab.tsx`) shows **project-create fields** and explicitly says listing / bonds / licenses are not extracted

Design choice: One extra Claude tool in the **same** spec job, after or beside goals — do not run a second full-PDF job. Page selection must include non-goal pages (bonds, listing, LDs).

4.1 — Page selection: more than DBE language

Files

- `worker/src/select-spec-pages.ts`
- Tests: `worker/src/select-spec-pages.test.ts` (run from the `worker` package)

Current

`GOAL_TERMS` , `LINK_TERMS` , `hasNoGoalLanguage` short-circuits with **no Claude call** if no goal terms. After this change, “no DBE language” must **not** skip the job if bid-requirement terms exist.

Add `REQUIREMENT_TERMS` (examples)

`bid bond` , `proposal guarantee` , `bidder's bond` , `liquidated damage` , `working day` , `calendar day` , `prevailing wage` , `dir` , `contractor's license` , `class a` , `listing` , `public contract`

code, 4104, subcontractor list, engineer's estimate, engineers estimate, bid opening, bid date, must be received

Scoring

Include pages that hit requirement terms even with 0 goal hits. Raise `MAX_SELECTED_PAGES` only if tests show the bond clause is dropped (try 32 before 48).

Rename / clarify `hasNoGoalLanguage`

e.g. `hasNoExtractableLanguage` = no goal AND no requirement AND no link terms. Update callers in `extract-participation-goals.ts` that skip Claude.

Done when

A fixture page with only “liquidated damages of \$2,500 per calendar day” is selected; a page of Standard Specs boilerplate is not.

4.2 — Types for extracted requirements

Add in **both** `worker/src/types.ts` and `lib/cost-engine/types.ts`:

```
export type ExtractedBidRequirement = {
  key:
    | "bid_date"
    | "engineers_estimate"
    | "license_class"
    | "bid_bond_percent"
    | "listing_threshold"
    | "working_days"
    | "liquidated_damages"
    | "prevailing_wage"
    | "other"
  rawText: string
  value?: string // as printed, not normalized to a Date unless obvious
  confidence?: number
  sourcePage?: number
  notes?: string
}
```

Add `bidRequirements?: ExtractedBidRequirement[]` on `TakeoffResult`.

Keep `participationGoals` / `specLinks` as they are.

4.3 — Extractor: second tool or expanded prompt

Preferred: same file `extract-participation-goals.ts` (or rename later to `extract-specifications.ts`):

1. Keep `record_participation_goals` as now.
2. Add tool `record_bid_requirements` with the checklist fields.
3. Same message can `tool_choice` auto with both tools, **or** two sequential `messages.create` calls on the **same selected pages** (simpler to parse; costs ~2× — acceptable for v1).

Prompt rules (copy the goals extractor’s anti-hallucination)

- Only what is printed.
- Empty array if the excerpt has no such clause.
- Never fill “typical 10% bid bond.”
- `rawText` verbatim.
- `listing_threshold` is the printed % or dollar rule, not legal advice.
- If the document is not specs, empty arrays + `documentNotes` .

`process-job.ts` specifications branch

Attach `bidRequirements: extracted.requirements` on `result` . `itemCount` can stay `goals.length` or use `goals.length + requirements.length` .

Usage kind

Keep `participation_goals_extraction` or add `spec_requirements_extraction` if you make a second API call — then `worker/src/ai-limits.ts` price table must include the model (same `TAKEOFF_MODEL`).

4.4 — App: read requirements in Intelligence

`app/intelligence/actions.ts`

- New `BidRequirementView` : key, label, value, rawText, notes, documentName, sourcePage, confidence.

- Map `job.result.bidRequirements` like goals.
- Pass into `IntelligenceShell` → `BidRequirementsTab`.

`components/intelligence/bid-requirements-tab.tsx`

- Table: Requirement | Extracted value | Source (reuse `SourceChip`).
- Under each row, muted verbatim `rawText`.
- If a field is missing from extraction, fall back to project fields **only** for: bid date, working days, LDs, prevailing wage, engineer's estimate (already on `project`). Label fallback **From project setup, not the specs**.
- Remove or rewrite the footer that says listing / bonds aren't extracted — replace with **Confirm against the specs. Constimator does not decide bid/no-bid**.
- Low confidence (`< 90`): warning badge.

Do **not** add a Bid / Pass button or score.

4.5 — Tests

- `select-spec-pages.test.ts` : requirement-only page selected; TOC-only “see section 2 for bonds” maybe low score — add a case.
 - Optional: parse fixture for `record_bid_requirements` shape (unit-test a mapper, not live Claude).
 - Real docs: one Notice to Bidders + one Special Provisions through `scripts/real-job/upload.ts --type specifications`. Check bond %, listing, LDs, PW, goal against the PDF by hand.
-

4.6 — Honesty / demo traps

- `components/intelligence/risks-tab.tsx` : keep real projects on “Risk detection isn't available yet.” Do not enable Shasta RFIs for other jobs (`project.number === demoProject.number` stays).
 - `components/features.tsx` : reads DBE/DVBE and a bid/no-bid **checklist** (bond, license, listing, time, LDs) from the specs — you confirm. It does not tell you whether to bid.
-

List 4 — out of scope

- “Should we bid?” model
 - Live RFI generation from plans
 - Auto-writing `projects.bidDate` without a confirm (optional later: “Apply to project” per field — skip in v1)
-

Appendix A — Suggested Cursor order (one chat per line)

1. List 1.1 empty state
2. List 1.2 import preview skip + confidence
3. List 1.3 confidence on bid table
4. List 1.4 seed estimate button
5. List 1.6 copy
6. List 1.7 tests
7. List 1.5 processing next-step
8. List 1.8 three real PDFs (you + worker, not only the agent)
9. List 2.1–2.3 plan holders
10. List 2.5 compare disclaimer
11. List 2.4 sidebar groups
12. List 2.6 summary card
13. List 3.1 migrate → 3.2 email → 3.3 action → 3.4 UI
14. List 4.1 pages → 4.2 types → 4.3 worker → 4.4 UI → 4.5 real spec book

When ready to build, open Cursor and say **start 1.1** (or whichever id). Implement only that checkbox.

Appendix B — Key file index

Area	Files
Bid-form extract	<code>worker/src/extract-bid-form.ts</code> , <code>worker/src/process-job.ts</code>
Bid-form import	<code>lib/bid-form-import.ts</code> , <code>lib/bid-form-import.test.ts</code>
Reconciliation UI	<code>components/reconciliation/reconciliation-shell.tsx</code> , <code>bid-form-import-card.tsx</code> , <code>bid-line-table.tsx</code>
Reconciliation actions	<code>app/reconciliation/actions.ts</code>
Estimate seed	<code>app/estimate/actions.ts</code> → <code>importFromBidScheduleAction</code>
Bid schema	<code>db/schema.ts</code> table <code>bids</code> (<code>extractionConfidence</code>)
Plan holders	<code>app/plan-holders/actions.ts</code> , <code>components/plan-holders/review-shell.tsx</code> , <code>holder-row.tsx</code>
Quote compare	<code>lib/quote-comparison.ts</code> , <code>components/sub-quotes/comparison-grid.tsx</code>
Spec goals	<code>worker/src/extract-participation-goals.ts</code> , <code>worker/src/select-spec-pages.ts</code>
Intelligence	<code>app/intelligence/actions.ts</code> , <code>components/intelligence/bid-requirements-tab.tsx</code>
Email	<code>lib/email/client.ts</code> , <code>lib/email/review-request.ts</code>
Scoped DB	<code>lib/db/scoped.ts</code>
Real-job harness	<code>scripts/real-job/README.md</code>
Registry (do not build)	<code>docs/REGISTRY-SOURCES.md</code>

Appendix C — Honest reliability (for client conversations)

Feature	Achievable version	Reliability
Bid-form → recon	Upload → spot-check → import	High on clean digital civil/DOT PDFs; confirm click is required
Who to list	Plan holders + leveled quotes	Assistive. Not a legal listing answer
Bid invites	Email confirmed holders, log sent	Not AI. Risk is missing emails on agency rosters
Spec bid/no-bid	Confirmable checklist of printed facts	Same class as table transcription when clauses are clearly printed. Never a bid/no-bid verdict

End of playbook. Implement List 1 before anything else.